CS 535: Assignment 04

Programming Assignments (90%)

- **This assignment is in C++.**
- Copy src/app/Assign03.cpp and name it **src/app/Assign04.cpp**
- Change the application name from "Assign03" to "Assign04"
- Change the window title to be "Assign04: " + your SITNET ID
    - E.g., "Assign04: realemj"
- Make a copy of the vulkanshaders/Assign03 folder and name it **vulkanshaders/Assign04**
- Modify **CMakeLists.txt** by adding the following line to the end of the file:
    - CREATE_VULKAN_EXECUTABLE(Assign04)
- Make sure the program configures, compiles, and runs as-is

Assign04.cpp

- Add the following structs definitions:
    - ```
      struct UBOVertex {
        alignas(16) glm::mat4 viewMat;
        alignas(16) glm::mat4 projMat;
      };
      ```
    - ```
      struct alignas(16) PointLight {
        glm::vec4 pos;
        glm::vec4 color;
      };
      ```
    - ```
      struct alignas(16) UBOFragment {
        glm::vec4 cameraPos;
        alignas(16) uint32_t lightCnt;
      };
      ```
    - ```
      struct Camera {
        glm::vec3 eye = glm::vec3(0.0f, 0.0f, 1.5f);
        glm::vec3 center = glm::vec3(0.0f, 0.0f, 1.0f);
        glm::vec3 up = glm::vec3(0.0f, 1.0f, 0.0f);
      };
      ```

- Add the following globals:
    - **Camera camera {};**
    - **UBOVertex uboVertHost {};**
    - **UBOFragment uboFragHost {};**
    - **vector<PointLight> hostLights {};**
    - **glm::vec2 lastMousePos {};**
    - **bool lightDataChanged = false;**

- Set the following:
    - **float zOffset = 0.0f;**
    - **bool flipWindingOrder = false;**

- Change the mouse motion callback **mouse_position_callback()**:
    - Get the current mouse position in pixels and get the DIFFERENCE in mouse movement:
        - **glm::vec2 mousePos = glm::vec2(xpos, ypos);**
        - **glm::vec2 diffPos = lastMousePos - mousePos;**
    - Store the last known mouse position (IN PIXELS): **lastMousePos = mousePos;**
    - Get the current framebuffer size:
        - **int width, height;**
        - **glfwGetFramebufferSize(window, &width, &height);**
    - If the width and height are both greater than zero:
        - Normalize the mouse motion difference:
            - **diffPos.x /= width;**
            - **diffPos.y /= height;**
        - Multiply by speed:
            - **float speed = 50.0f;**
            - **diffPos *= speed;**
        - Compute the current camera direction and local X axis:
            - **glm::vec3 direction = glm::normalize(camera.center - camera.eye);**
            - **glm::vec3 localX = glm::normalize(glm::cross(direction, camera.up));**
        - Create the appropriate matrices:
            - **glm::mat4 negEye = glm::translate(-camera.eye);**
            - **glm::mat4 rotateY = glm::rotate(glm::radians(diffPos.x), camera.up);**
            - **glm::mat4 rotateX = glm::rotate(glm::radians(diffPos.y), localX);**
            - **glm::mat4 posEye = glm::translate(camera.eye);**
        - Transform the camera "look at" point by 1) moving it by -eye, 2) rotating around the up axis, 3) rotating it around the local X axis, and 4) moving it back by +eye:
            - **camera.center = glm::vec3(posEye*rotateX*rotateY*negEye*glm::vec4(camera.center, 1.0));**

- Modify key callback function:
    - At the beginning of the function, compute the camera direction, the local X direction, and the speed:
        - **glm::vec3 direction = glm::normalize(camera.center - camera.eye);**
        - **glm::vec3 localX = glm::normalize(glm::cross(direction, camera.up));**
        - **float speed = 0.1f;**
    - Add the following keys:
        - **GLFW_KEY_W:**
            - Add direction*speed to camera.eye AND camera.center
        - **GLFW_KEY_S:**
            - Subtract direction*speed from camera.eye AND camera.center
        - **GLFW_KEY_D:**
            - Add localX*speed to camera.eye AND camera.center
        - **GLFW_KEY_A:**
            - Subtract localX*speed from camera.eye AND camera.center
        - **GLFW_KEY_1:**
            - Change ALL light colors to white
            - Make sure to change lightDataChanged to true
        - **GLFW_KEY_2:**
            - Change ALL light colors to red
            - Make sure to change lightDataChanged to true
        - **GLFW_KEY_3:**
            - Change ALL light colors to green
            - Make sure to change lightDataChanged to true
        - **GLFW_KEY_4:**
            - Change ALL light colors to blue
            - Make sure to change lightDataChanged to true
        - **GLFW_KEY_X:**
            - Increment max light count (making sure it is does not exceed hostLights.size())
        - **GLFW_KEY_Z:**
            - Decrement max light count (making sure it does not go less than 1)

- Modify **extractMeshData():**
    - Change the vertex colors to yellow: **glm::vec3(1,1,0,1)**

- Change **recordFrame()**:
    - Add the following parameters:
        - **pro::VulkanBuffer &uboVertData,**
        - **pro::VulkanBuffer &uboFragData,**

        - **pro::VulkanBuffer &ssboLights,**
        - **pro::DescriptorPack &descPack**
    - RIGHT BEFORE the call to **renderSceneNode():**
        - Compute the updated view and projection matrices based on the current camera information:
            - **uboVertHost.viewMat = glm::lookAt(camera.eye, camera.center, camera.up);**
            - **float fov = glm::radians(90.0f);**
            - **float aspectRatio = ((float) vkInitData.swapchain().extent.width) / ((float) vkInitData.swapchain().extent.height);**
            - **float near = 0.01;**
            - **float far = 1000.0;**
            - **uboVertHost.projMat = glm::perspective(fov, aspectRatio, near, far);**
        - Update the camera position for the fragment shader UBO:
            - **uboFragHost.cameraPos = glm::vec4(camera.eye, 1.0f);**
        - Copy over the UBO data:
            - **pro::copyToHostVisibleVulkanBuffer(vkInitData, uboVertData, &uboVertHost);**
            - **pro::copyToHostVisibleVulkanBuffer(vkInitData, uboFragData, &uboFragHost);**
        - IF the light data has changed, copy over the data to the SSBO and set lightDataChanged to false.
        - Bind descriptor sets:
            - **cd.commandBuffer.bindDescriptorSets(vk::PipelineBindPoint::eGraphics, pipelineData.layout, 0, descPack.sets, {});**

- In your **main()** function:
    - Hide the cursor:
        - **glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);**
    - Initialize the last mouse position:
        - **double xpos, ypos;**
        - **glfwGetCursorPos(window, &xpos, &ypos);**
        - **lastMousePos = glm::vec2(xpos, ypos);**
    - BEFORE pipeline creation, create the appropriate descriptor set layouts for the two uniform buffers and an SSBO:
        - **vector<vk::DescriptorSetLayoutBinding> allBindings = { vk::DescriptorSetLayoutBinding(0, vk::DescriptorType::eUniformBuffer, 1, vk::ShaderStageFlagBits::eVertex),**

```
vk::DescriptorSetLayoutBinding(1, vk::DescriptorType::eUniformBuffer, 1,
  vk::ShaderStageFlagBits::eFragment),

vk::DescriptorSetLayoutBinding(2, vk::DescriptorType::eStorageBuffer, 1,
  vk::ShaderStageFlagBits::eFragment)
};
```

- **pipelineCreateInfo.allDescSetLayouts.push_back(**
  vkInitData.device().createDescriptorSetLayout(
  vk::DescriptorSetLayoutCreateInfo({}, allBindings)));

- Create the appropriate UBOs for the vertex and fragment shader:
  - **pro::VulkanBuffer uboVertData = pro::createVulkanBuffer(vkInitData, sizeof(UBOVertex), vk::BufferUsageFlagBits::eUniformBuffer, pro::createVMAHostVisibleInfo());**
  - **pro::VulkanBuffer uboFragData = pro::createVulkanBuffer(vkInitData, sizeof(UBOFragment), vk::BufferUsageFlagBits::eUniformBuffer, pro::createVMAHostVisibleInfo());**
- Create a ring of 4 lights and add them to hostLights:
  - The y coordinate will always be 1.5f.
  - The radius is 1.5f.
  - The x coordinate should be from radius*cos()
  - The z coordinate should be from radius*sin()
  - Initial color should be white.
- Set initial lightCnt in uboFragHost to 1.
- Create the necessary SSBO for the lights and do the initial copy:
  - **pro::VulkanBuffer ssboLights = pro::createVulkanBuffer(vkInitData, sizeof(PointLight)*hostLights.size(), vk::BufferUsageFlagBits::eStorageBuffer, pro::createVMAHostVisibleInfo());**
  - **pro::copyToHostVisibleVulkanBuffer(vkInitData, ssboLights, hostLights.data());**
- Create the necessary descriptor pool sizes:
  - **vector<vk::DescriptorPoolSize> poolSizes = {**
    vk::DescriptorPoolSize(vk::DescriptorType::eUniformBuffer, 2),
    vk::DescriptorPoolSize(vk::DescriptorType::eStorageBuffer, 1)
    };
- Create the pro::DescriptorPack and update the descriptors:
  - **pro::DescriptorPack descPack = pro::createDescriptorPack(vkInitData, pipelineData, poolSizes);**
  - **pro::updateBufferForDescriptorPack(vkInitData, descPack, uboVertData, 0, 0);**
  - **pro::updateBufferForDescriptorPack(vkInitData, descPack, uboFragData, 0, 1);**

        - **pro::updateBufferForDescriptorPack(vkInitData, descPack,**
          ssboLights, 0, 2,
          vk::DescriptorType::eStorageBuffer);
    - Change the call to **recordFrame()** to include the new parameter values.
    - In the cleanup region:
        - **pro::cleanupVulkanBuffer(vkInitData, uboVertData);**
        - **pro::cleanupVulkanBuffer(vkInitData, uboFragData);**
        - **pro::cleanupVulkanBuffer(vkInitData, ssboLights);**
        - **pro::cleanupDescriptorPack(vkInitData, descPack);**

shader.vert

- Add the following structs:
    - **layout(set = 0, binding = 0) uniform UBOVertex {**
      mat4 viewMat;
      mat4 projMat;
      } ubo;

- Add new output variables:
    - **layout(location = 1) out vec3 interPos;**
    - **layout(location = 2) out vec3 interNormal;**

- In the main function:
    - Change interColor to take the original vertex color:
        - **interColor = color;**
    - Pass along the transformed normal to fragment shader:
        - **mat3 normalMat = transpose(inverse(mat3(pc.modelMat)));**
        - **interNormal = normalMat*normal;**
    - Compute the position after the model transform and ensure it is passed along to the fragment shader:
        - **vec4 pos = vec4(position, 1.0);**
        - **pos = pc.modelMat*pos;**
        - **interPos = vec3(pos);**
    - Compute the final position after projection:
        - **gl_Position = ubo.projMat*ubo.vertMat*pos;**

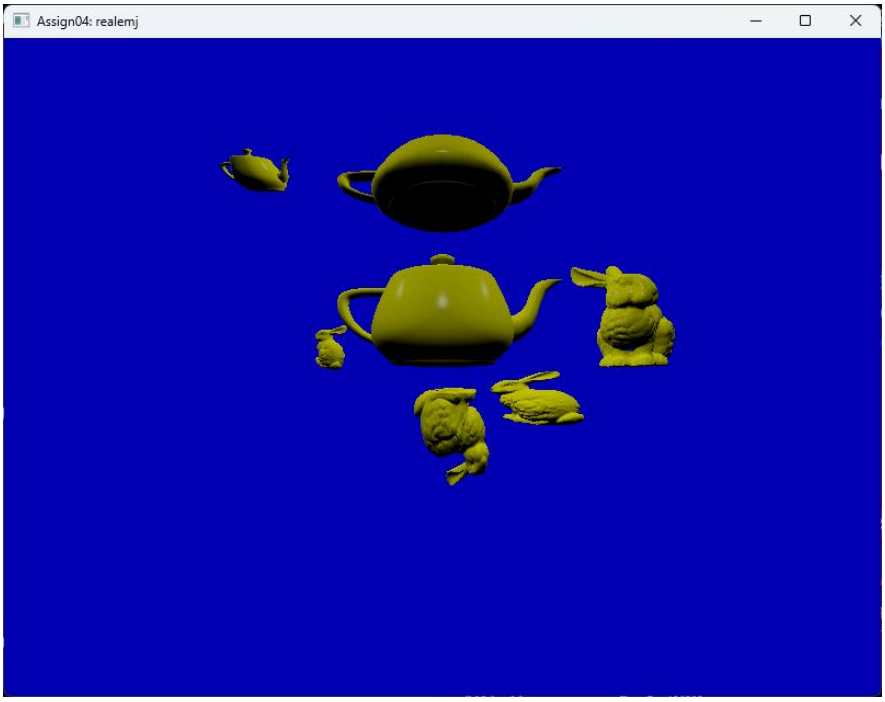
shader.frag

- Add the following structs:
    - **struct PointLight {**
      vec4 pos;
      vec4 color;
      };

    - **layout(set = 0, binding = 1) uniform UBOFragment {**
      vec4 cameraPos;
      uint lightCnt;
      } ubo;
    - **layout(set = 0, binding = 2, std430) readonly buffer SSBOLights {**
      PointLight allLights[];
      };
- Add new input variables:
    - **layout(location = 1) in vec3 interPos;**
    - **layout(location = 2) in vec3 interNormal;**
- In the main function:
    - Renormalize normal:
        - **vec3 N = normalize(interNormal);**
    - Create a final color vector:
        - **vec3 finalColor = vec3(0,0,0);**
    - For each light:
        - Compute the light vector:
            - **vec3 L = normalize(vec3(allLights[i].pos) - interPos);**
        - Compute the diffuse component:
            - **float diffuseComp = max(0, dot(N, L));**
        - Calculate the diffuse lighting:
            - **vec3 diffuseColor =**
              vec3(allLights[i].color)*vec3(interColor)*diffuseComp;
        - Compute the (normalized) view vector:
            - **vec3 V = normalize(vec3(ubo.cameraPos) - interPos);**
        - Compute the half vector:
            - **vec3 H = normalize(L+V);**
        - Compute the specular component (with a shininess of 500):
            - **float specComp = pow(max(0, dot(N,H)), 500);**
        - Compute the specular color:
            - **vec3 specColor = vec3(allLights[i].color)*specComp*diffuseComp;**
        - Add both to final color:
            - **finalColor += diffuseColor + specColor;**
    - Perform Reinhard tone mapping:
        - **finalColor = (finalColor)/(finalColor + vec3(1.0f,1.0f,1.0f));**
    - Output the final color:
        - **out_color = vec4(finalColor, 1.0);**

Screenshots (5%)

For this part of the assignment, **upload ONE screenshot** of the application window
(note that window title should be "Assign04: yoursitnetid") with the **bunnyteatime.glb** model and **all 4 lights active**:



Copy this image as "**Assign04.png**" into the **screenshots** folder.

README (5%)

- Under the "Applications" section, add a subsection (same format as Assign01) very briefly describing your Assign04 code.

Grading

Your OVERALL assignment grade is weighted as follows:

- 90% - Programming
- 5% - Screenshots
- 5% - README

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	Maximum Penalty
Code that does not compile	60
Fragment shader incorrect	20
Vertex shader incorrect	15
Camera controls incorrect	15
Lights incorrect	10
Light count/color controls incorrect	10
Wrong initial mouse	5
Vertex color not yellow	5
Technically correct code, but poor code design	5
Missing assignment Git branch	5
Missing screenshots	5
README update missing/incorrect	5

In general, these are things that will be penalized:

- **Code that does not compile (up to 60 points off!)**
- Sloppy or poor coding style
- Bad coding design principles
- Code that crashes, does not run, or takes a VERY long time to complete
- Using code from ANY source other than the course materials
- Collaboration on code of ANY kind; this is an INDIVIDUAL PROJECT
- Sharing code with other people in this class or using code from this or any other related class
- Output that is incorrect
- Algorithms/implementations that are incorrect
- Submitting improper files
- Failing to submit ALL required files